UNIVERSITY OF
CAMBRIDGE

Title

Elisabeta-Iulia (Julia) Dima

King's College

This dissertation is submitted on June, 2026 for the degree of Master of Philosophy

Abstract

My abstract ...

Declaration

This dissertation is the result of my own work and includes nothing which is the outcome of work done in collaboration except as declared in the Preface and specified in the text. It is not substantially the same as any that I have submitted, or am concurrently submitting, for a degree or diploma or other qualification at the University of Cambridge or any other University or similar institution except as declared in the Preface and specified in the text. I further state that no substantial part of my dissertation has already been submitted, or is being concurrently submitted, for any such degree, diploma or other qualification at the University of Cambridge or any other University or similar institution except as declared in the Preface and specified in the text. This dissertation does not exceed the prescribed limit of 60 000 words.

Elisabeta-Iulia (Julia) Dima
June, 2026

Acknowledgements

My acknowledgements ...

Contents

Chapter 1

Introduction

Understanding what neural networks *compute*, not only what they *output*, is the central problem of mechanistic interpretability. A model that correctly solves arithmetic problems or chains reasoning steps coherently need not be implementing anything resembling the algorithm a human would use. Whether it does, where in the network that computation lives, and whether it can be manipulated predictably are empirical questions that evaluation benchmarks cannot answer.

1.1 Mechanistic Interpretability

As neural networks scale from toy models to systems executed in consequential and complex settings, the gap between performance and understanding becomes untenable. Thus, a model achieving near-human accuracy on reasoning benchmarks cannot be straightforwardly understood. A system that has learned a spurious shortcut is indistinguishable from one that has learned the intended algorithm until it fails at the margin. The interpretability methods that accompanied earlier machine learning, such as saliency maps, attribution scores, and probing classifiers, operate in the input or output space and offer correlational accounts but are not mechanistic, as they describe what the model attends to, not what it computes.

Evidence for the tractability of mechanistic questions accumulated independently. Work on convolutional vision networks identified low-level features, including edge detectors, curve analysers, and frequency filters, that arise independently of architecture and training seed [8]. Analysis of transformer attention heads revealed heads implementing specific, describable operations, such as induction heads completing repeated token sequences and positional heads copying information from fixed offsets [4]. A direct demonstration of such mechanisms is the grokking phenomenon [10], which shows that small models trained on modular arithmetic initially memorise their training data and later undergo a phase transition, developing compressed algorithms that generalise across all held-out operand pairs for the trained modulus. The transition is invisible to accuracy metrics evaluated on the training set. However, Fourier decomposition of the learned weight matrices reveals that the converged model implements modular arithmetic through specific trigonometric identities encoded in the residual stream, an algorithm with no surface similarity to the training

objective [7].

Further work has confirmed that small transformers trained on arithmetic tasks converge on specific, inspectable algorithms, independent of surface statistical regularities in the data [13, 11, 12, 6].

The same interpretability considerations apply to large language models (LLMs), yet at a scale that makes manual circuit analysis infeasible. As LLMs are trained on vastly more diverse data and without explicit algorithmic curricula, they nonetheless achieve high accuracy on structured reasoning tasks. However, whether this reflects stable, localisable internal computation or brittle surface associations cannot be determined from evaluation alone. This distinction matters wherever model outputs yield real consequences.

Anthropic's *On the Biology of a Large Language Model* [5] proposes a reproducible methodology for approaching interpretability on a large language model (LLM). Combining sparse autoencoders (SAEs) with cross-layer transcoders (CLTs) and backward-pass attribution, it constructs dependency graphs that trace information flow from input tokens through intermediate features to output logits. The framework forms the methodological foundation of this thesis, which examines how structured reasoning is implemented within large instruction-tuned models and what the geometry of those representations reveals about the nature of computation.

1.2 Interpretability Methods

Let $\mathcal{M}$ be a transformer with L layers, residual stream dimension d, and MLP sublayers $\mathrm{MLP}^{(0)}, \ldots, \mathrm{MLP}^{(L-1)}$. All methods employed in this thesis operate on the residual stream of $\mathcal{M}$. Let $\mathbf{h}_t^{(\ell)} \in \mathbb{R}^d$ denote the residual-stream vector at layer $\ell \in \{0, \ldots, L-1\}$ and token position t. Each layer contributes an additive update,

$$\mathbf{h}_t^{(\ell+1)} = \mathbf{h}_t^{(\ell)} + \mathrm{Attn}^{(\ell)}\left(\mathbf{H}^{(\ell)}\right)_t + \mathrm{MLP}^{(\ell)}\left(\mathbf{h}_t^{(\ell)}\right), \tag{1.1}$$

where $\mathbf{H}^{(\ell)}$ denotes the full sequence of residual-stream vectors at layer ℓ. This additive structure makes the residual stream a natural object for linear analysis, as each layer writes into a shared representation that accumulates across depth.

Mechanistic analysis replaces each MLP sublayer $\mathrm{MLP}^{(\ell)}$ with a pretrained *transcoder* $\mathcal{T}_\ell$, a sparse encoder–decoder whose weights are fit offline to reconstruct the MLP output while decomposing it into K interpretable directions [1, 3]. For the input $\mathbf{x} \in \mathbb{R}^d$ to the MLP sublayer at layer ℓ, the transcoder computes pre-activations

$$z_{\ell,k}(\mathbf{x}) = \left(\mathbf{W}_{\mathrm{enc}}^{(\ell)}\,\mathbf{x} + \mathbf{b}_{\mathrm{enc}}^{(\ell)}\right)_k, \tag{1.2}$$

so that feature k carries activation $a_{\ell,k}(\mathbf{x}) = \mathrm{ReLU}(z_{\ell,k}(\mathbf{x}))$, and the reconstructed MLP output is

$$\hat{f}_\ell(\mathbf{x}) = \mathbf{W}_{\mathrm{dec}}^{(\ell)}\,\mathbf{a}_\ell(\mathbf{x}) + \mathbf{b}_{\mathrm{dec}}^{(\ell)}, \tag{1.3}$$

where $\mathbf{W}_{\text{dec}}^{(\ell)} \in \mathbb{R}^{d \times K}$ and $\mathbf{a}_\ell(\mathbf{x})$ is the vector of all feature activations at layer ℓ. Features are indexed throughout by the double subscript (ℓ, k), with ℓ specifying the layer and k the index within that layer's dictionary. Sparsity ensures that at most a small active set contributes for any given input, making each feature's role inspectable in isolation.

Attribution scores are computed on the *linearised* model, in which attention outputs are detached and treated as constants, so the residual stream propagates solely through the transcoder reconstruction path. This linearisation is the necessary condition for the scores to be additive and interpretable as influence flows through the network. Let T denote the position of the final input token, and let $y_j = \mathbf{v}_j^\top \mathbf{h}_T$ be the logit for output token j, where $\mathbf{h}_T$ is the pre-unembedding hidden state and $\mathbf{v}_j$ is the demeaned unembedding direction for token j. The attribution score of feature k at position t in layer ℓ is

$$\alpha_{\ell,k,t} = a_{\ell,k,t} \cdot \frac{\partial y_j}{\partial a_{\ell,k,t}}, \tag{1.4}$$

the product of the feature activation with the gradient of the target logit with respect to that activation [5]. Collecting all such scores over a given prompt yields a directed weighted graph, the *attribution graph*, in which nodes are (layer, position, feature) triples and edges carry the influence $\alpha_{\ell,k,t}$ from earlier features to later ones. The graph is pruned by retaining only those nodes and edges that account for a fixed fraction of the total downstream influence on y_j, controlled by a node threshold τ_{node} and an edge threshold τ_{edge}.

Activation patching establishes causal claims [2]. Given a clean prompt x and a counterfactual prompt x', patching replaces the residual stream at layer ℓ and position t,

$$\tilde{\mathbf{h}}_t^{(\ell)} = \mathbf{h}_t'^{(\ell)}, \tag{1.5}$$

and measures the resulting shift in output logit, $\Delta_j = y_j(\tilde{\mathbf{h}}) - y_j(\mathbf{h})$. A non-zero Δ_j confirms that the patched representation is causally sufficient to alter the prediction. Constrained patching restricts the replacement to a targeted subset of positions or components, enabling finer localisation of causal dependence.

Steering vectors are extracted by contrasting two prompt sets, D_+ where a concept is present and D_- where it is absent, and computing their mean difference at a designated anchor position t^*,

$$\delta^{(\ell)} = \mathbb{E}_{x \in D_+}\left[\mathbf{h}_{t^*(x)}^{(\ell)}\right] - \mathbb{E}_{x \in D_-}\left[\mathbf{h}_{t^*(x)}^{(\ell)}\right]. \tag{1.6}$$

Under the linear representation hypothesis [9], $\delta^{(\ell)}$ approximates the axis along which the concept is encoded at depth ℓ. Injecting $\alpha\,\delta^{(\ell)}$ into the residual stream at inference time provides a continuous lever on the concept representation without modifying model weights. The norm profile $\|\delta^{(\ell)}\|$ across layers is the primary signal used for concept localisation throughout this thesis. In combination, these four methods support mechanistic claims about how a model computes, not only whether it succeeds on a given input.

1.3 This Thesis

We take the open-access model `Qwen3-4B` as our subject. The primary contribution is a systematic concept localisation study across a diverse set of concepts spanning three structural categories: modular arithmetic (carry propagation in integer addition, GCD divisibility, and residue class membership), relational reasoning (transitive ordering and negation scope), and physical and causal plausibility (energy conservation and causal direction). The central hypothesis is that modular concepts, whose outcomes are determined by discrete thresholds or periodic relations, produce qualitatively different localisation signatures from relational and physical concepts, whose outcomes depend on monotone comparison or world knowledge.

The rest of this dissertation is structured as follows. Chapter 2 covers the relevant background. .....

Chapter 2

Related Work

2.1 Arithmetic algorithms in transformers

A productive line of mechanistic interpretability research has asked not merely whether transformers can perform arithmetic, but *how* they do so: what internal representations they form and what computational steps those representations support.

Nanda et al. [7] studied a one-layer transformer trained on modular addition and found that grokking, the phenomenon whereby generalisation appears suddenly after extended training on memorised data, corresponds to the emergence of a Fourier-based algorithm. The trained model encodes operands as superpositions of trigonometric features and composes them across layers to compute the result. The emergence of this structure can be tracked through interpretable progress measures even before the accuracy curve improves significantly.

Zhong et al. [13] revisited modular addition and showed that the same task admits two qualitatively distinct algorithmic solutions: a Fourier-based strategy they term the "clock" algorithm, and a lookup-table strategy they term the "pizza" algorithm. Which solution emerges depends on training configuration, demonstrating that identical supervision can produce structurally different circuits and that mechanistic diversity is not solely an artefact of random initialisation.

For standard integer addition, Quirke and Barez [11] identified a carry-propagation circuit distributed across attention and MLP layers, bearing a structural resemblance to the long-addition algorithm familiar from elementary arithmetic. Quirke et al. [12] extended this account to subtraction, characterising how the same underlying circuit adapts to accommodate sign.

Musat [6] approached the problem from the perspective of training dynamics, tracing how the internal representations of operands and results cluster and align over the course of training on modular addition. Their analysis shows that organised algorithmic structure is not installed gradually but emerges through identifiable phase transitions, reinforcing the view that grokking marks a qualitative change in the model's internal organisation rather than a smooth accumulation of generalising evidence.

These results establish a strong prior that transformers trained specifically on arithmetic tasks do not merely interpolate over training examples but converge on discrete, inspectable

computational procedures. All of the models studied, however, are small transformers trained from scratch on a single arithmetic task. The present work asks whether comparable structure is recoverable in `Qwen3`, a multi-billion-parameter instruction-tuned model trained on a broad natural-language corpus with no explicit arithmetic curriculum.

Chapter 3

Reproducibility

3.1 Reproducing the Addition Arithmetic Circuit

Lindsey et al. [5] showed that Claude 3.5 Haiku, when presented with prompts of the form `calc: a+b=`, does not implement a symbolic carry algorithm but instead activates a family of sparse *lookup table* features whose firing concentrates on the cells of the operand space satisfying $a \bmod 10 = d_1$ and $b \bmod 10 = d_2$ for a specific digit pair (d_1, d_2). These features encode the ones digit of $d_1 + d_2$ and route this information directly to the output. The present reproduction asks whether the same mechanistic signature appears in Qwen3-4B, trained independently on a different corpus and with a different tokeniser, and whether the attribution methodology transfers faithfully to a model for which it was not originally calibrated. The analysis adopts the transcoder formalism and notation of Section 1.2; the faithfulness of the linearised attribution on Qwen3-4B is examined separately in Section **??**.

3.1.1 Operand Plots and Lookup Feature Identification

For a prompt `calc: a+b=` , let T be the index of the `=` token. The *operand matrix* for feature (ℓ, k) is

$$M_{\ell,k}[a, b] \;=\; a_{\ell,k,T}(\texttt{calc: } a\texttt{+}b\texttt{= }), \qquad a, b \in \{0, \ldots, 99\}, \tag{3.1}$$

obtained by sweeping over the full 100×100 integer-pair grid. The geometry of $M_{\ell,k}$ encodes the feature's sensitivity in a directly interpretable form. A feature that tracks the raw sum $a + b$ produces diagonal stripes of constant activation; one that tracks a single operand produces horizontal or vertical bands; modular sensitivity to the ones digit of either operand produces a period-10 repeating structure. A feature that fires selectively for a specific pair of ones digits (d_1, d_2) produces a regular grid of isolated point clusters, concentrated on the 200 cells where $a \bmod 10 \in \{d_1, d_2\}$ and $b \bmod 10 \in \{d_1, d_2\}$. These *lookup table features* constitute 2% of the operand space; a uniform feature would achieve the same coverage by chance.

To identify lookup candidates automatically, each feature is assigned the concentration score

$$s_{\ell,k}(d_1, d_2) = \frac{\sum_{a,b} M_{\ell,k}[a,b] \mathbb{1}[a \bmod 10 \in \{d_1, d_2\},\ b \bmod 10 \in \{d_1, d_2\}]}{\sum_{a,b} M_{\ell,k}[a,b]}, \tag{3.2}$$

which measures the fraction of total activation mass that lies on the target grid. Features are filtered by a minimum peak-activation threshold to exclude dormant features, then ranked by $s_{\ell,k}$. The top-ranked feature for the pair $(d_1, d_2) = (6, 9)$ – the ones digits of the focus operands 36 and 59 – is taken as the reference lookup feature and subjected to causal validation. Two interventions are applied. In the *inhibition* experiment, the feature's activation is scaled by a factor $\beta \in \{1.0, 0.5, 0.0, -1.0, -5.0, -10.0\}$ before the decoder output is computed; if the feature is causally necessary, driving β below zero should monotonically weaken the logit for the token representing the correct ones digit of $6 + 9 = 15$. In the *substitution* experiment, the MLP inputs at all layers preceding the lookup feature's layer are replaced by those from the prompt `calc: 39+59=` , which has ones digits $(9, 9)$ rather than $(6, 9)$; if the identified feature mediates the computation, the output should shift from predicting ones digit 5 to ones digit 8.

Position-resolved feature sweep. Lindsey et al. [5] observe that the features active during addition in Claude 3 Haiku differ systematically by token position: features at the operand tokens encode individual digit values, while features at the delimiter token encode their combination. To examine whether an analogous positional structure is present in Qwen3-4B, the operand sweep of Equation 3.1 was applied at three token positions in the prompt `calc:` a`+`b`=`: the ones digit of a (position 4), the ones digit of b (position 7), and the `=` token (position 8). At each position the top-15 CLT features by attribution edge weight were selected; one representative per distinct ones-digit encoded is shown. Figure 3.1 shows the resulting matrices.

At position 4, the selected features activate in vertical bands with period 10 along the a axis. Activation is determined almost entirely by a mod 10 and is insensitive to b. At position 7 the pattern is transposed: horizontal bands with period 10 in b, independent of a. The symmetry between the two positions suggests that the ones digits of the two addends are encoded separately, with each operand token carrying information about that operand's own digit.

The `=` token displays a more varied set of activation patterns. Several features produce diagonal bands concentrated along contours of constant $a + b$, indicating sensitivity to the sum rather than to individual operand values. Others show a periodic point-cluster structure in which activation concentrates at cells where both a mod 10 and b mod 10 take specific values; this is the joint-modular structure captured by the score in Equation 3.2, and it is the defining signature of the lookup table features described in Section 3.1.1. A smaller number of features at this position show broader diagonal regions reflecting coarser magnitude sensitivity rather than exact digit values.

The positional separation visible in Figure 3.1 implies that ones-digit information is encoded

before the two addends are jointly processed. At the operand token positions, no retained feature shows sensitivity to both a and b simultaneously; the cross-operand joint representation appears only at the `=` token. This ordering is consistent with the view that the network first reads off each digit independently and only subsequently computes a function of their combination.

3.1.2 Experimental Scope

The experiment is structured around four prompt collections, summarised in Table 3.1.

Table 3.1: Prompt collections used in the addition circuit reproduction. The operand grid drives the activation sweep; the focus prompt anchors the attribution graph.

Split	Template	N	Purpose
Operand grid	`calc: a+b=`	10,000	Feature activation sweep; $a, b \in \{0, \ldots, 99\}$
Focus prompt	`calc: 36+59=`	1	Primary attribution graph; answer `95`

The 10,000-prompt grid exhausts the integer-pair space $\{0, \ldots, 99\}^2$, covering all 100 distinct ones-digit combinations and providing sufficient support to identify modular activation patterns visually. The focus prompt `calc: 36+59=` follows Anthropic's exact choice, enabling direct comparison of attribution graphs across models.

3.1.3 Behavioural Verification

Before committing mechanistic resources to a single focus prompt, it is necessary to establish that the model solves the task reliably across the operand space. A circuit identified on a prompt the model cannot answer carries no mechanistic authority. The accuracy sweep therefore records, for every pair $a, b \in \{0, \ldots, 99\}$, the *teacher-forced confidence* at each output position. Let $\mathbf{y}^*(a, b) = (y_0^*, y_1^*, \ldots)$ denote the ground-truth answer tokens for $a + b$. The teacher-forced confidence at position k is

$$P_k(a, b) = P\Big(y_k = y_k^* \;\Big|\; y_0 = y_0^*, \ \ldots, \ y_{k-1} = y_{k-1}^*, \ \mathbf{x}(a, b)\Big), \tag{3.3}$$

where $\mathbf{x}(a, b)$ is the token sequence for the prompt `calc: a+b=` and preceding positions are forced to their correct tokens. Equation (3.3) isolates the residual difficulty at each generation step by removing uncertainty due to error accumulation in prior positions.

Figure 3.2 plots $P_k(a, b)$ for $k \in \{0, 1, 2\}$. At $k = 0$ the model must predict the leading output token from the prompt alone; the grid mean is $\mu_0 = 0.233$, reflecting genuine uncertainty over a substantial fraction of the operand space. At $k = 1$, once the first token is teacher-forced to the correct value, mean confidence rises to $\mu_1 = 0.943$. By $k = 2$ it reaches $\mu_2 = 0.988$, with the only uncertain region being the upper-right triangle where $a + b \geq 100$ and the answer carries into a third digit. The cascade reveals that the central computational burden falls entirely on predicting y_0^*: subsequent tokens resolve almost automatically given the correct leading digit, a pattern that

is consistent with a mechanism that maps directly from the operand ones-digits to the output rather than propagating carries sequentially.

Rather than varying smoothly with operand magnitude, the $k = 0$ panel of Figure 3.2 shows that confidence is organised into a period-10 grid aligned with the decimal ones-digit boundaries. Within each 10×10 block the within-block pattern repeats with the same geometry, and this periodicity implies that P_0 depends primarily on (a mod 10, b mod 10) rather than on the absolute values of a and b. This is the behavioural signature one would expect if the computation were implemented by features selective to specific ones-digit pairs.

Prediction entropy. Confidence $P_k(a, b)$ measures whether the top-ranked prediction is correct, but not how sharply the model has committed to any prediction. A cell with low confidence might correspond either to genuine distributional uncertainty or to a confident wrong prediction. The Shannon entropy of the truncated output distribution at position 0,

$$H(a, b) = -\sum_{j=1}^{K} p_{(j)}(a, b) \ \log_2 p_{(j)}(a, b), \tag{3.4}$$

where $p_{(1)}(a, b) \geq \cdots \geq p_{(K)}(a, b)$ are the top-K predicted probabilities, measures distributional spread independently of prediction correctness. Figure 3.3 plots $H(a, b)$ across the grid. Low-entropy cells (where the model commits decisively to a single token) form a period-10 grid aligned precisely with the high-confidence regions of the $k = 0$ panel of Figure 3.2. High-entropy cells (where the model distributes probability across several candidates) are concentrated at and near carry boundaries and for large operand values. The agreement between the two maps establishes that the model's decisiveness and its correctness are driven by the same underlying structure, rather than by one compensating for the other.

Carry complexity. Each pair (a, b) can be classified by the number of digit-column carries arising in the sum. Defining

$$\kappa(a, b) = \begin{cases} 0 & \text{no carry arises,} \\ 1 & \text{exactly one carry arises,} \\ 2 & \text{two or more carries arise,} \end{cases} \tag{3.5}$$

Figure 3.4 overlays the partition induced by κ on the confidence grid. If Qwen3-4B were implementing a symbolic carry algorithm, accuracy should deteriorate with carry complexity: pairs with $\kappa = 2$ should be hardest and pairs with $\kappa = 0$ easiest. The right panel of Figure 3.4 contradicts this prediction. Regions of identical carry class exhibit substantial internal variation in P_0, and the drops in confidence cut across carry boundaries rather than aligning with them. The period-10 grid structure of P_0 persists within each carry class, unmodulated by carry count. This dissociation between carry complexity and model accuracy constitutes behavioural evidence that the computation is not organised around carry propagation.

TODO: Conclusions...

3.2 Reproducing the Multilingual Circuit

Lindsey et al. [5] showed that antonym completion in Claude 3.5 Haiku is not implemented by a monolithic circuit but by three functionally independent components, each localised in a distinct portion of the network. When presented with a prompt of the form *the opposite of "small" is "*, the model activates a family of transcoder features encoding the antonym *operation* (what semantic relation is being applied), the *operand* (the concept whose opposite is sought), and the output *language* (which language the response should appear in). By identifying source-exclusive and target-exclusive features for each component in isolation and injecting the latter while suppressing the former, they demonstrated that each component can be redirected independently without disrupting the others. The present experiment asks whether the same factorised structure is present in Qwen3-4B, a model trained on a different corpus with a different tokeniser.

The multilingual setting makes the factorisation hypothesis non-trivial to test. If the operation and operand components were language-specific, features identified from an English prompt would carry no predictive power over French or Chinese outputs. The hypothesis predicts instead that the features encoding the antonym relation and the concept of size are language-neutral directions in activation space, remaining active across prompts in all three languages, and that only the language component varies with the surface language of the input. This thesis reproduces three causal interventions on Qwen3-4B, targeting the operation, operand, and language components in turn, using the transcoder-based causal methodology described below.

3.2.1 Exclusive Feature Identification

For each experiment, a *source* condition and a *target* condition are defined, differing along exactly one of the three components. Let $a_{\ell,k}^{\text{src}}$ and $a_{\ell,k}^{\text{tgt}}$ denote the activation of transcoder feature k at layer ℓ and token position p under the source and target conditions respectively. Taking the top-K active features in each condition, a feature is *source-exclusive* if it is strongly active in the source condition and weakly active in the target, and *target-exclusive* if the reverse holds. With exclusivity threshold $\rho \in (0, 1)$, the suppress set $\mathcal{S}^\ell$ and inject set $\mathcal{I}^\ell$ at layer ℓ are defined as

$$\mathcal{S}^\ell = \left\{ k \in \mathcal{F}_{\text{src}}^\ell : a_{\ell,k}^{\text{tgt}} < \rho \cdot a_{\ell,k}^{\text{src}} \right\}, \qquad \mathcal{I}^\ell = \left\{ k \in \mathcal{F}_{\text{tgt}}^\ell : a_{\ell,k}^{\text{src}} < \rho \cdot a_{\ell,k}^{\text{tgt}} \right\}, \tag{3.6}$$

where $\mathcal{F}_{\text{src}}^\ell$ and $\mathcal{F}_{\text{tgt}}^\ell$ are the top-K active feature indices at layer ℓ under the respective conditions. The threshold $\rho = 0.5$ was used throughout, so a feature qualifies as source-exclusive only if the target condition activates it at less than half the source-condition magnitude. The sets $\mathcal{S}^\ell$ and $\mathcal{I}^\ell$ are disjoint by construction: a feature active at comparable magnitude in both conditions satisfies neither criterion and belongs to neither set.

3.2.2 Causal Intervention Sweep

The intervention acts on the pre-MLP hidden state. Let $\mathbf{h}^{\ell,p} \in \mathbb{R}^d$ denote the post-RMSNorm input to the MLP at layer ℓ and token position p, the vector on which the transcoder encoder was trained to operate. For each layer ℓ in the designated range, $\mathbf{h}^{\ell,p}$ is projected through the transcoder encoder, the resulting feature representation is modified according to Equation (3.6), and the altered output supplants the standard MLP contribution at that position.

The feature activations produced by this projection are

$$\tilde{\mathbf{z}}^{\ell,p} = \mathrm{ReLU}\Big(\mathbf{W}_{\mathrm{enc}}^{(\ell)}\,\mathbf{h}^{\ell,p} + \mathbf{b}_{\mathrm{enc}}^{(\ell)}\Big) \in \mathbb{R}^K. \tag{3.7}$$

Source-exclusive activations are set to zero and target-exclusive activations are augmented at intervention strength $\gamma \geq 0$,

$$\tilde{z}_k^{\ell,p} \;\leftarrow\; 0, \quad k \in \mathcal{S}^\ell; \qquad \tilde{z}_k^{\ell,p} \;\leftarrow\; \tilde{z}_k^{\ell,p} + \gamma\, v_k, \quad k \in \mathcal{I}^\ell, \tag{3.8}$$

where $v_k = a_{\ell,k}^{\mathrm{tgt}}$ is the target-condition activation magnitude of feature k. The modified feature vector is then projected back to residual space via the transcoder decoder,

$$\tilde{\mathbf{o}}^{\ell,p} = \mathbf{W}_{\mathrm{dec}}^{(\ell)}\,\tilde{\mathbf{z}}^{\ell,p} + \mathbf{b}_{\mathrm{dec}}^{(\ell)}, \tag{3.9}$$

and computation propagates with $\tilde{\mathbf{o}}^{\ell,p}$ in place of the standard MLP output at position p and layer ℓ, all other positions and layers remaining undisturbed. The next-token probability for each monitored vocabulary index t at the final position T is

$$\pi_t(\gamma) = \Big[\mathrm{softmax}(\mathrm{logits}_T)\Big]_t. \tag{3.10}$$

The sweep is conducted over $\gamma \in \{0,\, \delta,\, 2\delta,\, \ldots,\, \gamma_{\max}\}$, yielding one probability curve π_t per monitored token. If the targeted component is causally sufficient for the corresponding aspect of the output, π_t for the target-condition token should rise monotonically with γ while the source-condition curve declines, constituting a directional redistribution of probability mass.

3.2.3 Three Causal Interventions

The three interventions differ in which component they target, at which token position features are identified, and over which layer range the modification is applied. The layer ranges are grounded in the decomposition reported by Lindsey et al. [5] for Claude 3.5 Haiku: the operation component occupies the middle third of the network, the operand the early third, and the language component the first fifth. For Qwen3-4B with $L = 36$, these correspond to

$$\mathcal{L}_{\mathrm{op}} = \Big\{\Big\lfloor \tfrac{L}{3} \Big\rfloor, \ldots, \Big\lfloor \tfrac{2L}{3} \Big\rfloor - 1\Big\} = \{12, \ldots, 23\}, \tag{3.11}$$

$$\mathcal{L}_{\text{oper}} = \left\{0, \ldots, \left\lfloor \tfrac{L}{3} \right\rfloor - 1\right\} = \{0, \ldots, 11\}, \tag{3.12}$$

$$\mathcal{L}_{\text{lang}} = \left\{0, \ldots, \max\left(1, \left\lfloor \tfrac{L}{5} \right\rfloor\right) - 1\right\} = \{0, \ldots, 6\}. \tag{3.13}$$

These ranges are adopted from Anthropic's analysis of a distinct architecture and are not derived from the geometry of Qwen3-4B itself. Whether they correctly delimit the relevant layer segments for this model is an empirical question that the results address.

Operation swap. The antonym task is presented in English as *the opposite of "small" is "*, with the alternative prompt *a synonym of "small" is "* as the target condition. Features are identified at the final token position $p = T$ over layers $\mathcal{L}_{\text{op}}$. The resulting exclusive partition $(\mathcal{S}^\ell, \mathcal{I}^\ell)$ is transferred without modification to antonym prompts in English, French, and Chinese. The monitored tokens are the antonym surface forms in each language and their synonym counterparts. If the operation features are language-neutral, the English-derived partition should redirect the output toward synonyms across all three languages.

Operand swap. The source condition is the English antonym prompt for "small" and the target is the English antonym prompt for "hot", *the opposite of "hot" is "*. Features are identified not at the final position but at the *operand divergence position* p^*, the first index at which the token sequences of the two English prompts diverge. This is the position at which the words "small" and "hot" enter the context respectively, and it is where the operand concept is hypothesised to be encoded. The exclusive partition is computed over $\mathcal{L}_{\text{oper}}$ and applied at the analogous divergence position in each language-specific antonym prompt. The monitored tokens are the antonym surface forms and the surface forms denoting cold in each language.

Language swap. Three directed language pairs are examined: English to Chinese, Chinese to French, and French to English. For each pair, both the source and target activations are drawn from antonym prompts in the respective languages, and the exclusive partition encodes the contrast between two language contexts rather than two semantic conditions. The partition is computed at the final position over $\mathcal{L}_{\text{lang}}$ and the intervention acts on the source-language prompt. The monitored tokens are the antonym surface forms in both languages. If the language component is independently localised, substituting target-language features for source-language ones should redirect the output toward the target-language antonym, leaving the underlying semantic relation undisturbed.

3.2.4 Results

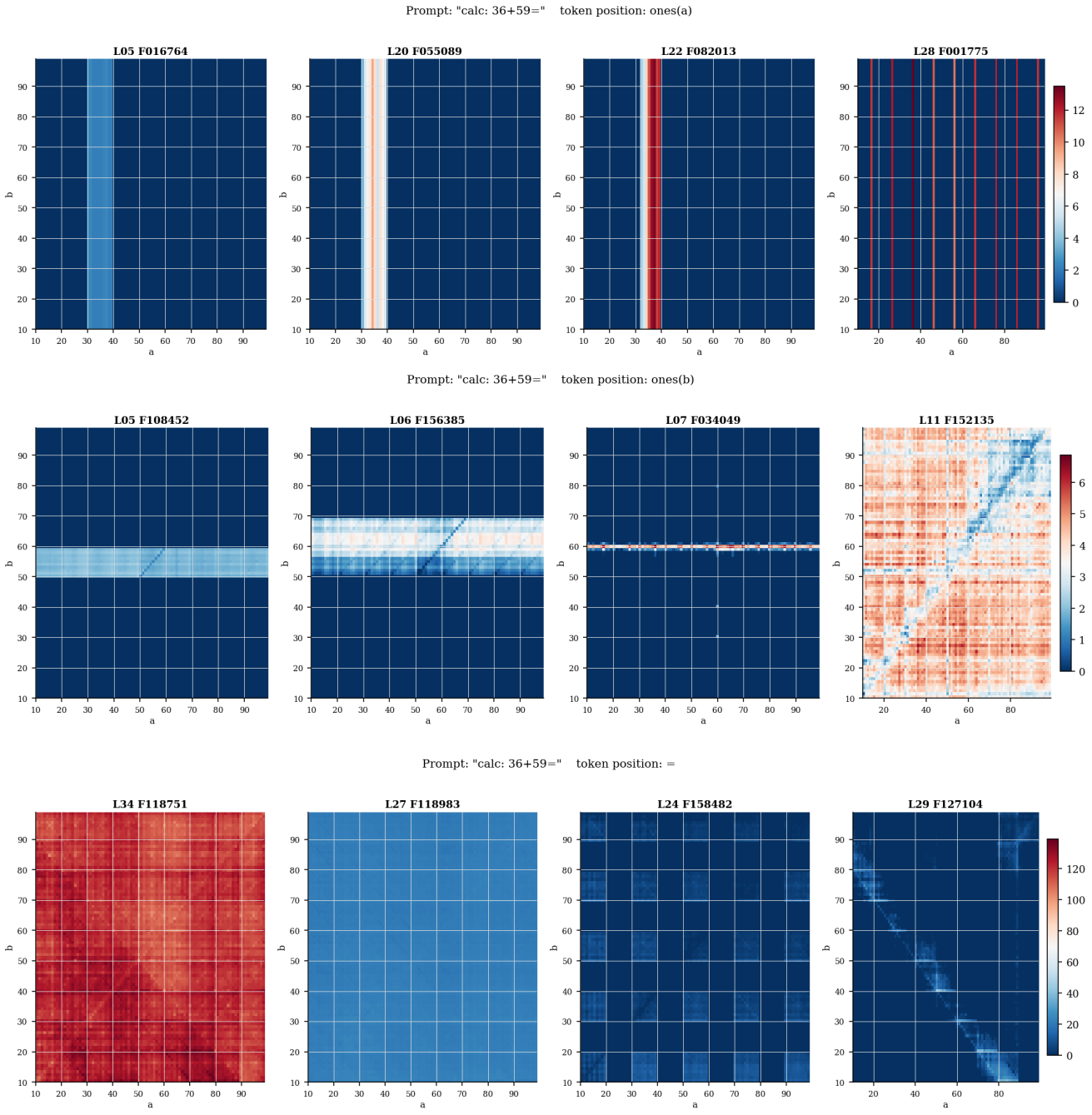

Figure 3.1: Representative operand matrices $M_{\ell,k}[a, b]$ at three token positions in `calc: 36+59=`, one per distinct ones-digit encoded. *Top:* features at the ones digit of a (position 4); each panel activates along a single vertical stripe determined by a mod 10, and is flat in b. *Middle:* features at the ones digit of b (position 7); the pattern is transposed, with activation concentrated in a single horizontal stripe determined by b mod 10. *Bottom:* features at the `=` token (position 8), showing periodic point-cluster grids of joint-modular sensitivity and broader diagonal regions of sum sensitivity. Axes span two-digit operands $a, b \in [10, 99]$.

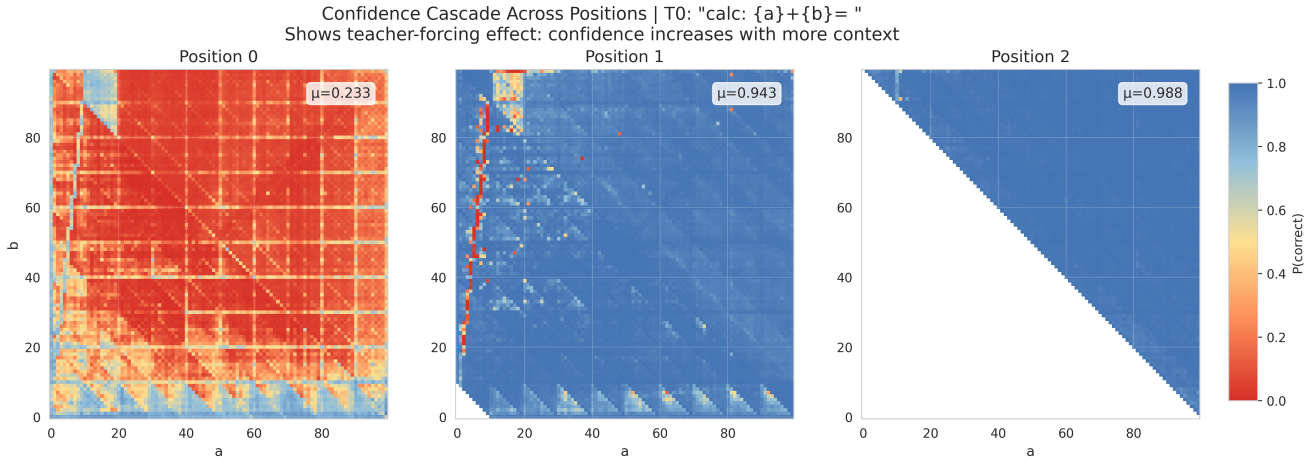

Figure 3.2: Teacher-forced confidence $P_k(a, b)$ (Eq. 3.3) across the 100×100 operand grid for positions $k = 0, 1, 2$. Each cell encodes the probability that Qwen3-4B assigns to the correct k-th output token when all preceding tokens are forced to their ground-truth values. Grid means are $\mu_0 = 0.233$, $\mu_1 = 0.943$, and $\mu_2 = 0.988$. The leftmost panel reveals a period-10 grid structure aligned with the decimal ones-digit boundaries: columns and rows separated by ten units carry systematically different baseline confidence levels, implying that P_0 depends primarily on (a mod 10, b mod 10) rather than on operand magnitudes. Empty cells in the middle panel correspond to pairs with $a + b < 10$, whose single-digit sum has no second output token; empty cells in the rightmost panel correspond to pairs with $a + b < 100$, whose two-digit sum has no third digit.

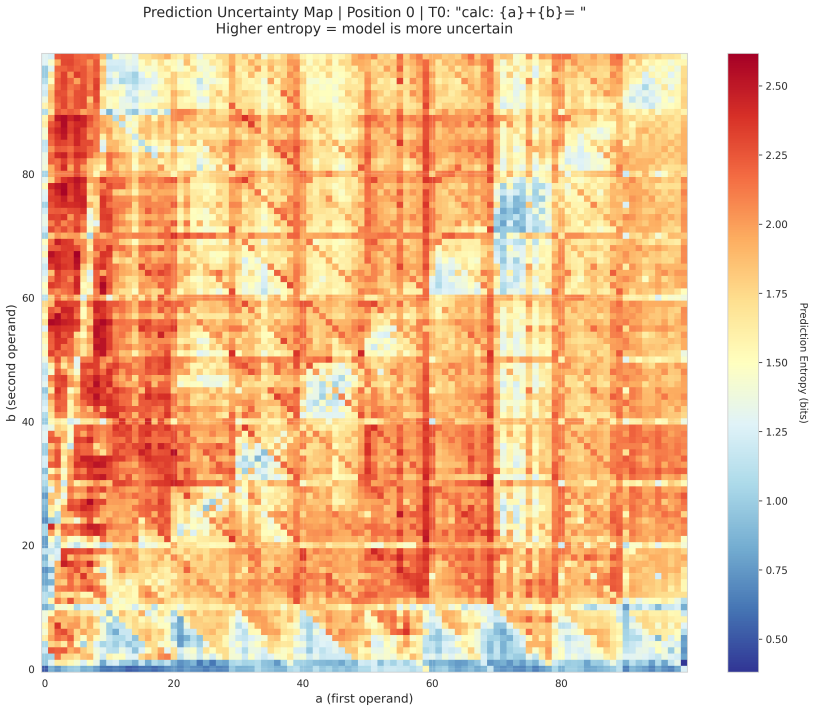

Figure 3.3: Prediction entropy $H(a, b)$ (Eq. 3.4) at position $k = 0$. Low-entropy cells (blue) form a period-10 grid aligned with the high-confidence cells visible in the $k = 0$ panel of Figure 3.2, indicating that the model is both decisive and correct on precisely the pairs where lookup features activate strongly. High-entropy cells (red) concentrate near carry boundaries and for large operand values, reflecting genuine distributional indecision rather than merely confident errors.

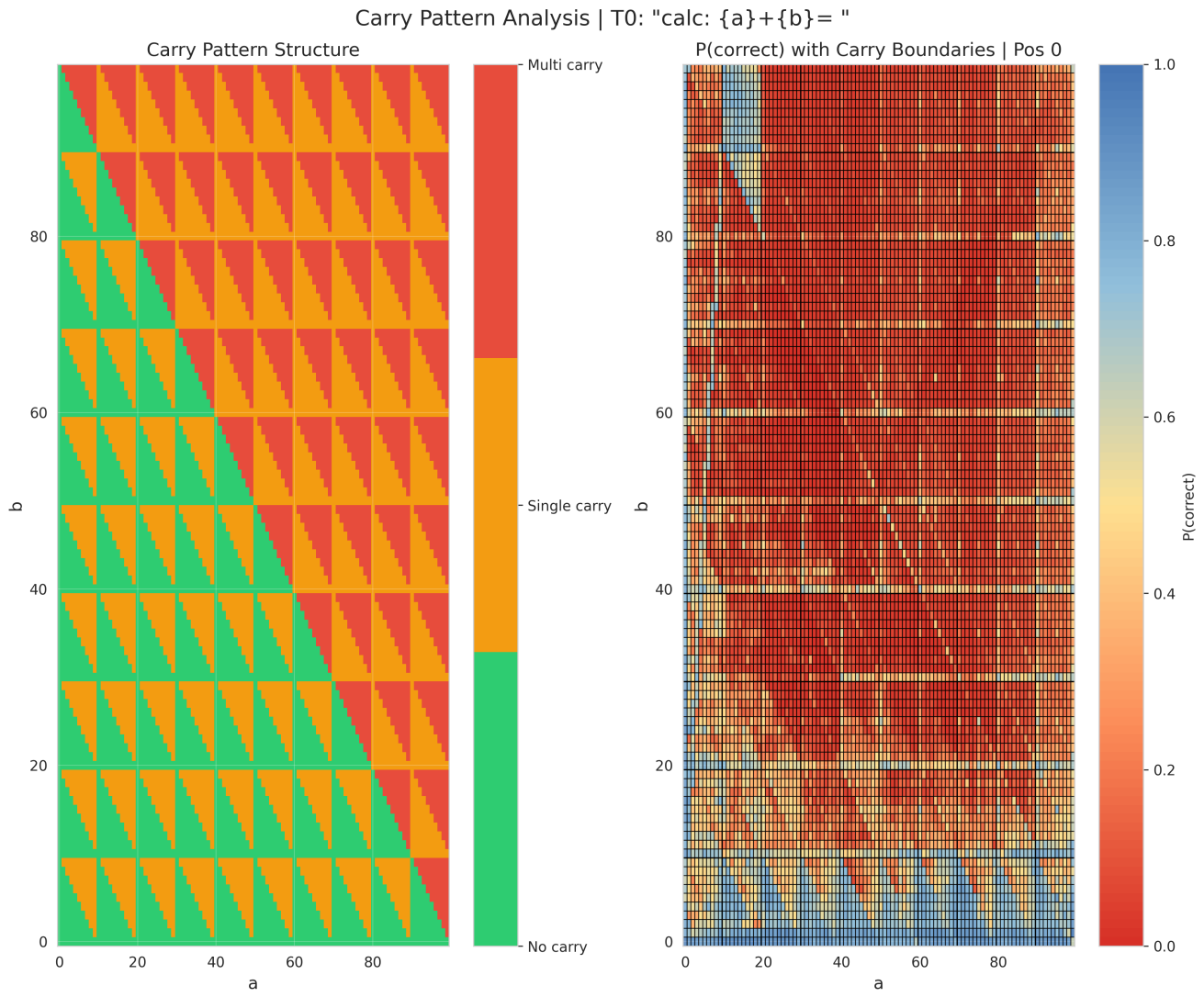

Figure 3.4: Left: carry-class partition $\kappa(a, b)$ (Eq. 3.5) of the operand grid, coloured by carry count. Right: first-token confidence $P_0(a, b)$ with carry-boundary contours superimposed. The period-10 structure of P_0 persists within each carry class, and drops in confidence do not align with the no-carry/single-carry/multi-carry boundaries, ruling out a symbolic carry algorithm as the underlying mechanism.

Chapter 4

Concept Localisation via Contrastive Residual-Stream Analysis

This chapter describes the contrastive localisation pipeline used to identify where arithmetic and symbolic concepts are represented inside Qwen3. The aim is not only to find a layer at which a concept is linearly separable, but also to connect that separation to sparse transcoder features and to causal effects on the model's answer. The method therefore combines four views of the same object: residual-stream deltas, feature-decoder alignment, activation patterns over structured datasets, and intervention scores.

4.1 Contrastive Pair Construction

Each concept is represented by a set of matched prompt pairs (x_i^+, x_i^-). The positive prompt contains the target property and the negative prompt differs in the smallest controlled way that removes it. Matching is important: if the two prompts differed in many incidental details, the mean residual stream difference would mostly recover those nuisance changes rather than the concept. The contrastive design attempts to hold syntax, length, operand scale, and answer format fixed while changing the mathematical property of interest.

For the carry dataset, a positive example is an addition problem whose units digits cross a carry boundary, while the negative example is chosen so that no carry is required. For example, a pair may contrast

`calc: 631+349=` with `calc: 638+556=`,

where the exact operands are sampled from the dataset generator. For greatest common divisor and residue-class datasets, the positive and negative examples are instead grouped by modular or divisor-related labels. These datasets are useful because the concept is not tied to a single token string: it is a relation between numbers.

The experiments use several surface templates. Template `T0` is the compact arithmetic form, such as `calc: a+b=`. Other templates use more verbal or question-like phrasing. Since anchor

Concept	Positive contrast	Controlled negative contrast
Carry	units digits require a carry	matched addition without carry
Residue class	target congruence relation holds	different residue class
GCD	target divisor relation holds	different gcd/divisor offset

Table 4.1: Examples of the contrastive structure used in the localisation pipeline. The actual datasets contain many sampled pairs per template; the table records the property that changes within a pair.

positions are not necessarily aligned across templates, per-anchor analyses are run within a single template. Cross-template plots are used only for comparisons at semantically matched anchors, such as the position corresponding to an operand digit. This avoids mixing residual-stream vectors from positions that do not play the same computational role.

4.2 Layer-Wise Concept Deltas

Let $h_{l,t}(x) \in \mathbb{R}^d$ denote the residual-stream vector at layer l and token position t for prompt x. Each pair has an anchor position t^*, chosen to be the token position at which the relevant information is available. In the carry experiments these anchors correspond to operand and answer positions such as `rank5_pos9`; in modular datasets they correspond to the positions at which the relevant number or answer token is represented.

For each layer, the concept delta is the mean positive residual stream minus the mean negative residual stream at the anchor:

$$\delta_l = \frac{1}{N}\sum_{i=1}^{N} h_{l,t^*}(x_i^+) - \frac{1}{N}\sum_{i=1}^{N} h_{l,t^*}(x_i^-). \tag{4.1}$$

Equivalently, if $\Delta h_{i,l} = h_{l,t^*}(x_i^+) - h_{l,t^*}(x_i^-)$, then $\delta_l = N^{-1}\sum_i \Delta h_{i,l}$. This form makes clear that the delta is not a single-example patch vector. It is an average direction over a distribution of matched contrasts.

Under the linear representation hypothesis [9], δ_l approximates an axis along which the concept is encoded at depth l, provided that the contrast is well controlled and the concept is approximately linearly separable. The delta should therefore be interpreted as a first-order summary of a representation, not as proof that the model uses a single one-dimensional variable. Several metrics are computed from δ_l to test whether this summary is geometrically stable, feature-aligned, and causally relevant.

4.3 Delta Metrics and Anchor Selection

The most direct statistic is the norm trajectory

$$D_l = \|\delta_l\|_2. \tag{4.2}$$

A peak in D_l means that positive and negative examples are most separated at that layer for the chosen anchor. In practice, norm trajectories are used as an initial localisation signal: early peaks often reflect lexical or template-specific processing, while mid-to-late peaks are more likely to reflect an abstract variable used by the computation.

Because different anchors can have different absolute scales, a normalised trajectory is also plotted:

$$\tilde{D}_l = \frac{D_l}{\max_j D_j + \varepsilon}. \tag{4.3}$$

The raw and normalised curves answer different questions. The raw curve shows the actual geometric scale of separation. The normalised curve shows where an anchor's own separation is concentrated, independent of its magnitude relative to other anchors.

A sharpness score summarises how concentrated the delta norm is across layers. In these experiments it is computed as a peak-to-average statistic,

$$\text{sharp}(t^*) = \frac{\max_l D_l}{\frac{1}{L}\sum_{l=0}^{L-1} D_l + \varepsilon}. \tag{4.4}$$

High sharpness indicates that the concept appears in a narrow processing window. Low sharpness indicates either diffuse representation or weak separation. Anchors are selected by combining peak norm, sharpness, and downstream causal scores. This prevents the analysis from selecting an anchor merely because it has a large but uninformative residual-stream scale.

The inter-layer cosine similarity matrix measures whether the delta direction is stable across layers:

$$C_{l,m} = \frac{\delta_l \cdot \delta_m}{\|\delta_l\|_2 \, \|\delta_m\|_2 + \varepsilon}. \tag{4.5}$$

Block structure in this heatmap indicates a processing phase in which the same concept direction persists. Rapid sign changes or isolated high-norm layers suggest that the model may be transforming the representation rather than maintaining a single stable code.

4.4 Feature-Space Interpretation

Residual-stream deltas are dense vectors in $\mathbb{R}^d$. To connect them to interpretable components, the pipeline projects each delta onto transcoder decoder features. For layer l, let $W_l^{\text{dec}} \in \mathbb{R}^{d \times F_l}$ be the transcoder decoder matrix, and let $w_{l,f}^{\text{dec}}$ be its f-th column. This vector is the residual-stream effect contributed when feature f is active.

The decoder cosine score is

$$\cos_{l,f}^{\mathrm{dec}} = \frac{w_{l,f}^{\mathrm{dec}} \cdot \delta_l}{\|w_{l,f}^{\mathrm{dec}}\|_2 \, \|\delta_l\|_2 + \varepsilon}. \tag{4.6}$$

A high positive value means that when the feature writes to the residual stream, it writes in approximately the same direction as the contrastive concept delta. A high negative value means that the feature writes in the opposite direction. This is not the same as saying that the feature's activation grid visually resembles the concept: a sparse feature can have high decoder alignment if its effect vector points along δ_l, even if it is active on only a few examples. For this reason, feature interpretation uses both decoder alignment and activation statistics.

For a selected feature, the activation grid is formed by averaging feature activations over arithmetic variables. In the carry case the grid is

$$A_f(a, b) = \mathbb{E}\left[z_{l,f}(x) \mid a(x) \bmod 10 = a,\ b(x) \bmod 10 = b\right], \tag{4.7}$$

where $z_{l,f}(x)$ is the transcoder feature activation. The resulting 10×10 matrix can reveal whether a decoder-aligned feature is active on a carry boundary, an iso-sum band, a parity pattern, or a more local arithmetic condition. For intrinsic feature selection, the pipeline also records the fraction of dataset examples on which a feature is active. Requiring nontrivial activity, for example at least ten percent of examples, filters out features whose high cosine is dominated by one or two isolated cells.

Two overlap metrics are useful when comparing activation grids. The centred cosine between two grids A and B is

$$\mathrm{gridcos}(A, B) = \frac{\langle A - \bar{A}, B - \bar{B} \rangle}{\|A - \bar{A}\|_2 \, \|B - \bar{B}\|_2 + \varepsilon}, \tag{4.8}$$

which ignores the mean activation level and measures pattern similarity. A Jaccard score is computed after thresholding each grid to its active cells:

$$J(A, B) = \frac{|\operatorname{supp}(A) \cap \operatorname{supp}(B)|}{|\operatorname{supp}(A) \cup \operatorname{supp}(B)|}. \tag{4.9}$$

Cosine is sensitive to graded shape; Jaccard is sensitive to support overlap. The pipeline uses both because a smooth Fourier-like feature and a sharp detector can have similar support but different amplitudes, or similar amplitudes but different support.

4.5 Fourier Patterns in Arithmetic Feature Grids

Many arithmetic feature visualisations are functions on a digit torus,

$$a, b \in \{0, \ldots, 9\}, \qquad f(a, b) \in \mathbb{R}.$$

Fourier analysis gives a compact way to describe these grids. The natural basis is

$$\exp\Big(\frac{2\pi i}{10}(ua+vb)\Big), \qquad u,v \in \{0,\dots,9\}. \tag{4.10}$$

Every grid can be written as

$$f(a,b) = \sum_{u=0}^{9}\sum_{v=0}^{9} C_{u,v}\exp\Big(\frac{2\pi i}{10}(ua+vb)\Big). \tag{4.11}$$

For real-valued grids it is usually clearer to report cosine modes,

$$A\cos\Big(\frac{2\pi}{10}(ua+vb)+\phi\Big), \tag{4.12}$$

where $A = 2|C_{u,v}|$ and $\phi = \arg C_{u,v}$ for a non-DC conjugate pair.

The frequency indices have direct arithmetic interpretations:

Frequency pattern	Interpretation
(k,k)	depends on $a+b$
$(-k,k)$	depends on $b-a$
$(k,0)$	depends only on a
$(0,k)$	depends only on b
$(5,5)$	parity, $(-1)^{a+b}$

For example,

$$\cos\left(\frac{2\pi(a+b)}{10}\right)$$

is an iso-sum feature, while

$$\cos\left(\frac{2\pi(b-a)}{10}\right)$$

is an iso-difference feature. Parity is a single mode because

$$(-1)^{a+b} = \cos\left(\frac{2\pi(5a+5b)}{10}\right).$$

Not all arithmetic features are smooth. A carry-margin detector such as

$$\mathbf{1}[a+b=10]$$

is localised on a narrow diagonal boundary. It can still be represented in the Fourier basis, but it requires many coefficients. This distinction is useful: smooth sinusoidal grids indicate low-frequency arithmetic variables, while sharp boundaries indicate logical or threshold-like detectors assembled from several Fourier components.

In the plots, the top K Fourier approximation is

$$\hat{f}(a,b) = \mu + \sum_{j=1}^{K} A_j \cos\Big(\frac{2\pi}{10}(u_j a + v_j b) + \phi_j\Big). \tag{4.13}$$

The explained Fourier energy is

$$E_K = \frac{\sum_{j=1}^{K} |C_j|^2}{\sum_j |C_j|^2}. \tag{4.14}$$

A high E_1 means that the feature is close to a pure Fourier mode. A feature requiring many modes is more localised and should be read as a detector rather than as a single smooth arithmetic coordinate.

4.6 Symbolic Fits to Feature Activations

For selected carry features, symbolic regression is used as a second description of the activation grid. The input variables are engineered arithmetic quantities such as a, b, $a+b$, $|a-b|$, ab, $\mathrm{mod}(a+b,10)$, and carry-margin terms such as $|a+b-10|$. The target is the normalised feature activation $A_f(a,b)$. A fitted expression is accepted only as an interpretive summary of the grid, not as a claim that the model explicitly computes that formula.

The symbolic fit is evaluated with

$$R^2 = 1 - \frac{\sum_{a,b}\Big(A_f(a,b) - \hat{A}_f(a,b)\Big)^2}{\sum_{a,b}\Big(A_f(a,b) - \bar{A}_f\Big)^2}. \tag{4.15}$$

Low-complexity expressions are preferred. If a feature is a simple carry boundary, a short expression involving $|a+b-10|$ is more informative than a high-complexity formula with similar R^2. The Fourier approximation and symbolic fit are therefore complementary: Fourier describes periodic structure, while symbolic regression gives a compact arithmetic predicate when one exists.

4.7 Template Robustness

Template robustness tests whether a direction is tied to the model's computation or to the surface form of a prompt. For each template T, deltas are computed separately:

$$\delta_l^{(T)} = \mathbb{E}_{x\in\mathcal{D}_+^{(T)}}[h_{l,t_T^*}(x)] - \mathbb{E}_{x\in\mathcal{D}_-^{(T)}}[h_{l,t_T^*}(x)]. \tag{4.16}$$

The anchor t_T^* is template-specific. This is essential: token positions that share the same integer index across templates need not have the same semantic role. Consequently, single-anchor sweeps and causal plots are run within one template, usually `T0`. Cross-template comparisons are saved separately and compare semantically corresponding anchors, not raw token indices.

The cross-template alignment at a layer is

$$\text{align}_l(T, T') = \frac{\delta_l^{(T)} \cdot \delta_l^{(T')}}{\|\delta_l^{(T)}\|_2 \, \|\delta_l^{(T')}\|_2 + \varepsilon}. \tag{4.17}$$

High alignment near the peak layers suggests that the same concept direction emerges under different phrasings. Low alignment, especially early in the network, indicates that template-specific tokenisation and syntax still dominate the representation.

4.8 Causal Validation and Null Hypothesis Testing

The preceding metrics are representational. They show that a contrastive direction exists, but not that it is used to produce the answer. The pipeline therefore adds causal and null-hypothesis checks.

4.8.1 Activation Patching

Activation patching tests causal sufficiency. For each pair (x_i^+, x_i^-) and layer l, the positive residual vector at the anchor is inserted into the negative forward pass:

$$h_{l,t^*}(x_i^-) \leftarrow h_{l,t^*}(x_i^+).$$

Let $m(x) = \ell_{\text{pos}}(x) - \ell_{\text{neg}}(x)$ be the answer logit margin at the final prediction position. The patching score is

$$S_l^{\text{patch}} = \frac{1}{N} \sum_{i=1}^{N} \left[m\big(\tilde{x}_{i,[l]}^-\big) - m(x_i^-) \right], \tag{4.18}$$

where $\tilde{x}_{i,[l]}^-$ is the negative prompt with the layer-l anchor activation patched from the positive prompt. A large positive score means that the positive representation at that layer is sufficient to push the negative prompt toward the positive answer.

4.8.2 Gradient-Dot-Delta

Gradient-dot-delta is a first-order approximation to patching. For each negative prompt, the gradient of the margin with respect to the anchor residual stream is computed, and then dotted with the mean concept delta:

$$S_l^{\text{grad}} = \frac{1}{N} \sum_{i=1}^{N} \nabla_{h_{l,t^*}} m(x_i^-) \cdot \delta_l. \tag{4.19}$$

A Taylor expansion gives

$$m(\tilde{x}_{i,[l]}^-) \approx m(x_i^-) + \nabla_{h_{l,t^*}} m(x_i^-) \cdot \Big(h_{l,t^*}(x_i^+) - h_{l,t^*}(x_i^-) \Big). \tag{4.20}$$

Averaging over pairs connects the patching score to the gradient-dot-delta score. Agreement between the two curves suggests that the output is locally close to linear in the relevant residual-stream direction. Disagreement suggests higher-order effects or that the mean delta is not a faithful summary of the pairwise intervention.

The causal overlay plot places S_l^{patch}, S_l^{grad}, and the delta-norm trajectory on the same layer axis. The useful case is not merely a high delta norm, but a high delta norm that occurs near a positive causal score. This alignment indicates that the representation is both separable and used by the model's answer computation.

4.8.3 Null Permutation Test

A contrastive pipeline can produce apparent directions even when labels are unrelated to the concept. The null test estimates this background level by permuting positive and negative labels within the same template and recomputing the delta metrics. Let δ_l^π be the delta obtained under permutation π. The null distribution for a statistic q_l, such as $\|\delta_l\|$ or a causal score, is

$$\{q_l(\delta^\pi) : \pi = 1, \ldots, P\}\,. \tag{4.21}$$

The observed statistic is compared with the null mean and quantiles. A layer whose observed score is well outside the null range is less likely to reflect generic prompt variability. The null is computed per template so that template-specific position structure is not mixed into the baseline.

4.9 Interpreting the Full Per-Anchor Summary

For each selected anchor, the pipeline saves a vertically aligned summary over layers. The panels share the same layer axis so that changes can be read together: feature-decoder projection, raw and normalised delta norm, inter-layer cosine similarity, null comparison, and causal overlay. This layout is designed to answer a single practical question: at which layers is there a concept direction that is large, stable, feature-aligned, above the null baseline, and causally relevant?

The interpretation is deliberately conservative. A high decoder cosine without feature activation structure is only a direction match. A clean activation grid without causal effect is only a representation. A causal score without template robustness may be a surface-form artefact. The strongest evidence comes when these signals agree at the same anchor and nearby layers.

4.10 Results

The empirical results for each concept are reported using the metrics defined above. The main comparisons are between anchors, layers, and templates, with feature-level plots used to interpret the most aligned directions. The remaining analysis focuses on whether the arithmetic concepts

produce localised, reusable feature structure or whether they remain distributed across many weakly aligned components.

Bibliography

[1] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E. Burke, Tristan Hume, Shan Carter, Tom Henighan, and Chris Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. URL https://transformer-circuits.pub/2023/monosemanticity/index.html.

[2] Arthur Conmy, Augustine N. Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adrià Garriga-Alonso. Automated circuit discovery for mechanistic interpretability. In *Advances in Neural Information Processing Systems*, volume 36, 2023. doi: 10.48550/arXiv.2304.14997.

[3] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023.

[4] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. URL https://transformer-circuits.pub/2021/framework/index.html.

[5] Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Shan Carter, Tom Henighan, Adam Stevenson, Tom Conerly, Danny Hernandez, Adam Jermyn, Callum McDougall, et al. On the biology of a large language model. 2025. URL https://transformer-circuits.pub/2025/biology-of-a-lm/index.html. Anthropic Transformer Circuits Thread.

[6] Tiberiu Musat. Clustering and alignment: Understanding the training dynamics in modular addition, 2024.

[7] Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2301.05217.

[8] Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. Zoom in: An introduction to circuits. *Distill*, 2020. doi: 10.23915/distill.00024.001. URL https://distill.pub/2020/circuits/zoom-in.

[9] Kiho Park, Yo Joong Choe, and Victor Veitch. The linear representation hypothesis and the geometry of large language models, 2023.

[10] Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets, 2022.

[11] Philip Quirke and Fazl Barez. Understanding addition in transformers. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2310.13121.

[12] Philip Quirke, Clement Neo, and Fazl Barez. Understanding addition and subtraction in transformers, 2025.

[13] Ziqian Zhong, Ziming Liu, Max Tegmark, and Jacob Andreas. The clock and the pizza: Two stories in mechanistic explanation of neural networks. In *Advances in Neural Information Processing Systems*, 2023. doi: 10.48550/arXiv.2306.17844.

[14] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Hubert Li, Andeng Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, Shashwat Goel, Long Long, Michael J. Byun, Zifan Wang, Alex Mallen, Steven Basart, Sanmi Koyejo, Dawn Song, Matt Fredrikson, J. Zico Kolter, and Dan Hendrycks. Representation engineering: A top-down approach to ai transparency, 2023.

Glossary

Attribution graph A directed acyclic graph whose nodes represent model components (token embeddings, SAE or CLT features, attention heads, output logits) and whose edges represent gradient-weighted information flow between them. Introduced as a methodology by Lindsey et al. [5].

Circuit A subgraph of the model's computational graph comprising a small set of components (attention heads, MLP layers, or individual features) that together implement a specific behaviour. The term is used in mechanistic interpretability to denote minimal, interpretable sub-computations.

CLT — Cross-Layer Transcoder A sparse dictionary-learning module, analogous to a sparse autoencoder, that maps MLP inputs at one layer to residual-stream contributions, potentially spanning across layers.

FFN — Feed-Forward Network Synonym for MLP in the transformer literature.

Grokking The phenomenon whereby a neural network trained on a small dataset first memorises the training examples (near-perfect training accuracy, poor generalisation) and then, after many additional gradient steps, abruptly transitions to a generalising solution. Mechanistically characterised for modular arithmetic by Nanda et al. [7].

LLM — Large Language Model A neural language model with a parameter count large enough that emergent capabilities arise from scale alone, typically trained on broad natural-language corpora via next-token prediction.

MLP — Multilayer Perceptron In the transformer architecture, the feed-forward sub-layer of each block, consisting of two learned linear projections with a pointwise nonlinear activation in between. Despite the name, transformer MLPs typically have only a single hidden layer.

RMSNorm — Root Mean Square Layer Normalisation A variant of layer normalisation that rescales activations by their root mean square without subtracting the mean. Used in Qwen and other modern architectures in place of standard LayerNorm.

SAE — Sparse Autoencoder A dictionary-learning module trained to decompose neural network activations into a sparse linear combination of learned feature directions. Used in

mechanistic interpretability to identify human-interpretable features inside a model.

Lookup table feature A transcoder feature whose activation matrix $M_{\ell,k}[a, b]$ concentrates on the 200 cells satisfying $a \bmod 10 \in \{d_1, d_2\}$ and $b \bmod 10 \in \{d_1, d_2\}$ for a specific ones-digit pair (d_1, d_2). Such features encode the pre-computed result of the ones-digit sum $d_1 + d_2$ and form the mechanistic primitive of addition circuits identified by Lindsey et al. [5].

Operand matrix A 100×100 array $M_{\ell,k}[a, b]$ recording the activation of transcoder feature k at layer ℓ at the final input-token position, swept over all integer-pair prompts `calc: a+b=` with $a, b \in \{0, \ldots, 99\}$. The geometric pattern of this matrix encodes the quantity that the feature tracks.

Representation engineering A methodology for extracting and analysing concept directions in the activation space of a neural network by computing the mean difference in hidden-state vectors between semantically contrasting inputs. Proposed by Zou et al. [14] as a top-down approach to model transparency, complementary to the bottom-up circuit-based programme.

Sharpness index A scalar summary of how tightly a concept delta's norm is concentrated around its peak layer, defined as the fraction of total norm mass lying within a window of width three centred at the layer of maximum norm. Values close to one indicate a localised representation; values close to $3/L$ indicate a representation that is uniformly distributed across the network's L layers.

Activation patching A causal intervention technique in which a single intermediate representation, here the residual-stream vector at a chosen layer and token position, is replaced by the corresponding value from a different input. The resulting change in output logits quantifies the causal sufficiency of that representation for driving the model's prediction.

Gradient-dot-delta A first-order approximation to activation patching obtained by computing the inner product of the output-margin gradient with respect to a layer's residual stream, evaluated at the baseline input, and the concept delta vector at that layer. The approximation is exact when the output margin is linear in the residual stream; agreement with activation patching in practice confirms that the relevant neighbourhood is approximately linear.

Source-exclusive feature A transcoder feature that is strongly active under a source prompt condition and weakly active under a target condition, qualifying it for suppression during a causal intervention sweep. The complementary notion, a target-exclusive feature, is one that is strongly active in the target condition and weakly active in the source, qualifying it for injection. Formally, feature k at layer ℓ is source-exclusive if $a_{\ell,k}^{\text{tgt}} < \rho \cdot a_{\ell,k}^{\text{src}}$ for a threshold $\rho \in (0, 1)$.

Intervention sweep An experiment in which a causal modification of the model's internal activations is applied at a sequence of increasing strengths γ, and the resulting next-token

probabilities for a set of tracked tokens are recorded at each value. A successful sweep produces a monotone crossover between the source-condition and target-condition token probability curves, providing evidence that the modified features causally mediate the targeted aspect of the output.

Operand divergence position The index of the first token at which two prompt sequences differ, used in the operand swap experiment to locate the token position at which the operand concept enters the context. For the English prompts *the opposite of "small" is "* and *the opposite of "hot" is "*, this is the position of the tokens corresponding to "small" and "hot" respectively.